

From Apollo to AI: A Deep Look into State Space Models

Oshadha Samarakoon — e21345@eng.pdn.ac.lk

Abstract

This article explores the theoretical evolution of State Space Models (SSMs) from their origins in aerospace control systems to their modern application in efficient deep learning architectures like Mamba. By analyzing the parallel between Kalman Filtering and selective state updates in AI, we demonstrate how classical control theory offers a solution to the memory bottlenecks of Transformer models.

1 From Black Box to White Box: The Evolution of Control

In the history of engineering, few challenges have been as unforgiving as the Apollo program. It was not simply a matter of pointing a rocket at the moon and firing. It required guiding a fragile metal cylinder through a vacuum, managing rapidly changing fuel mass, and landing on a moving target, all with computing power weaker than a modern scientific calculator. Yet, it performed orbital mechanics in real time. This feat was achieved not through raw power, but through a fundamental breakthrough in control theory.

Standard control methods of the era relied heavily on **Transfer Functions**, mathematical tools that treat a system as a **Black Box**. These models mapped inputs (voltage) directly to outputs (speed) while ignoring the internal physics.

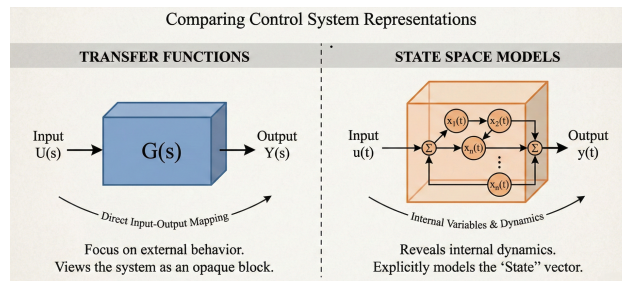


Figure 1: The shift in perspective. Transfer Functions (left) look only at Input/Output, while State Space Models (right) reveal the internal 'State' mechanism.

But for a rocket, the “internal physics” is everything. As fuel burns, mass drops. As altitude

changes, drag fluctuates. A simple input-output model fails because it cannot see inside the machine itself.

This necessity birthed the **State Space Model (SSM)**. Instead of a blind black box, Rudolf Kalman and his peers introduced a “White Box” approach [1]. They realized that to truly control a system, one must track its **State** (x), a hidden vector of internal variables like position, velocity, and pressure that evolves over time.

The turning point came in 1960, when Kalman visited NASA’s Ames Research Center. He presented his theory to Stanley Schmidt, a NASA engineer struggling with the Apollo trajectory problem. Schmidt realized that Kalman’s “linear filtering” could be adapted to estimate a spacecraft’s position even with noisy sensors; this breakthrough became the software heart of the Apollo Guidance Computer.

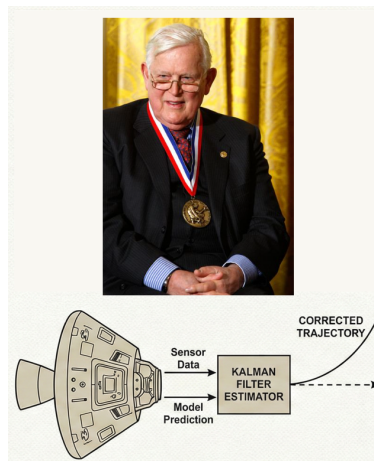


Figure 2: Rudolf Kalman, whose mathematical insights became the ‘invisible pilot’ of the Apollo missions.

For decades, this math remained the domain of aerospace and robotics. But today, the “Black Box” problem has returned in a new form: **Artificial Intelligence**. Just as Apollo engineers needed to track the internal state of a rocket, modern AI researchers are finding that “State Space” thinking is the key to handling massive, complex sequences of human language and streams of image data [1] [2] [3] [4] [5].

2 Looking into the State Space Model

To understand the power of Mamba, we must first strip away the “Black Box” of classical control and look at the internal mechanics of a State Space Model (SSM).

2.1 The Core Equations

Mathematically, a linear SSM describes a system using two clean equations: an update rule that evolves the hidden state, and an observation rule that maps that state to what we can see.

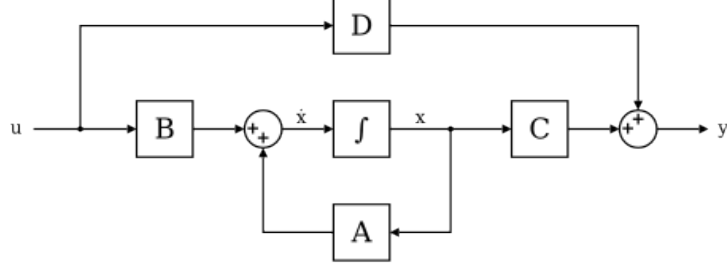


Figure 3: The Standard State Space Model. The integral symbol ($\int$) represents “memory”—storing the past to influence the future. [6]

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}u(t) \quad (\text{State Equation}) \quad (1)$$

$$y(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{D}u(t) \quad (\text{Observation Equation}) \quad (2)$$

Here, $\mathbf{x}(t)$ is the **Latent State Vector**, the internal memory of the system. In a mechanical system, this might be $[position, velocity]^T$. In a Language Model, this vector represents the semantic context of the sentence read so far.

2.2 From Rocket Physics to Digital Recursion

Consider a reusable rocket trying to land. We control the thruster force (u) to manage altitude (p). In the continuous world of physics, Newton’s laws give us a system where acceleration is proportional to force ($\dot{v} = u/m$).

However, computers and AI models cannot handle continuous time. They operate in discrete steps (Δt). To make this computable, we discretize the continuous system into a recursive update rule:

$$\mathbf{x}_{k+1} = \mathbf{A}_d \mathbf{x}_k + \mathbf{B}_d u_k$$

For a simple rocket model with a time step of $\Delta t = 1$, the transition looks like this:

$$\underbrace{\begin{bmatrix} p_{k+1} \\ v_{k+1} \end{bmatrix}}_{\mathbf{x}_{k+1}} = \underbrace{\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}}_{\mathbf{A}_d} \underbrace{\begin{bmatrix} p_k \\ v_k \end{bmatrix}}_{\mathbf{x}_k} + \underbrace{\begin{bmatrix} 0.5/m \\ 1/m \end{bmatrix}}_{\mathbf{B}_d} u_k$$

Notice the structure of Matrix A_d :

- The top row $[1, 1]$ implies $p_{k+1} = p_k + v_k$. We keep the old position and add velocity.
- The bottom row $[0, 1]$ implies $v_{k+1} = v_k$. Velocity persists unless acted upon.

2.3 The Recursive Advantage

Why does this matter for AI?

If we used standard calculus to find the position at $t = 100$, we would need to integrate the acceleration function from $t = 0$ to $t = 100$. This requires access to the entire history of inputs.

The State Space approach is **Recursive**. To find the state at step 100 (x_{100}), we do not need to revisit step 0. We only need the state at step 99 (x_{99}) and the current input (u_{99}).

$$x_{k+1} = f(x_k, u_k) \quad (3)$$

This property **constant memory cost** ($O(1)$) regardless of sequence length is the fundamental efficiency breakthrough. While Transformers (like GPT) must "look back" at every previous word to generate the next one ($O(N^2)$), an SSM-based model like Mamba simply carries its "State Vector" forward, updating it token by token.

3 State Estimation in Noisy Environments

In our previous simulation, we assumed perfect knowledge of the position. However, real-world systems operate in a "Reality Gap": mathematical models are ideal, but sensors (GPS, IMUs, Radar) are noisy.

3.1 The Observation Equation

To address this, the State Space Model introduces a second equation that tracks what we *see* versus what we *know*.

$$y_k = Cx_k + Du_k + v_k \quad (4)$$

Here, y_k is the **Observation** (e.g., GPS reading), and v_k represents **Measurement Noise**. Since we cannot see the true state x directly, we must infer it by filtering out v .

3.2 The Kalman Solution: Predict and Correct

Rudolf Kalman solved this with a recursive loop that mimics human intuition [1]. The filter balances the "Model Prediction" against the "Sensor Observation" using a weighting factor known as the **Kalman Gain** (K).

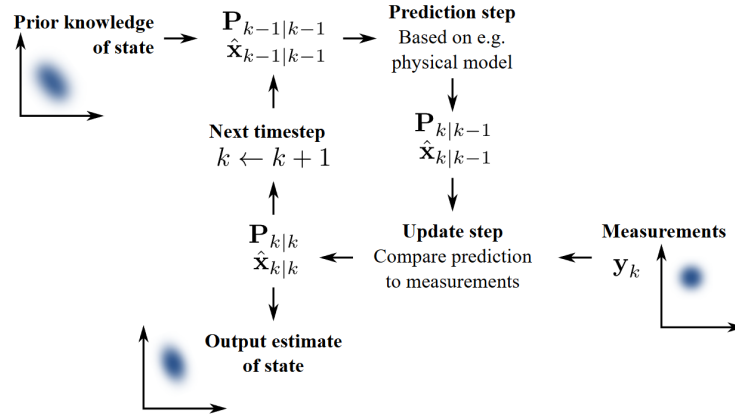


Figure 4: The Kalman Filter Cycle. The system predicts the next state based on physics, then corrects it based on sensor data. [7]

The cycle operates in two steps:

1. **Predict:** Use the system matrix (A) to estimate the next state. (e.g., “Thrust applied, expected altitude: 100m.”)
2. **Update:** Compare this prediction with the noisy sensor reading. (e.g., “Sensor reads: 110m.”)

The **Kalman Gain** (K) determines who to trust. It is a value between 0 and 1:

- **High K :** Trust the Sensor (Low noise).
- **Low K :** Trust the Model (High noise).

$$\text{New Estimate} = \text{Prediction} + K \times (\text{Observation} - \text{Prediction})$$

If our sensors are noisy ($K = 0.1$), the filter effectively ignores the “110m” reading and updates the estimate to just 101m, sticking close to the physics.

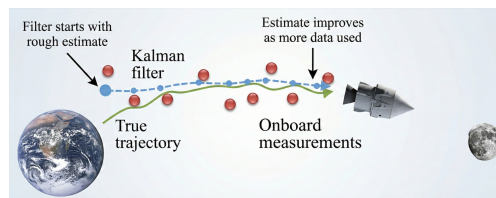


Figure 5: The Apollo Lunar Module trajectory. The onboard computer used Kalman Filtering to estimate this path in real-time. [8]

3.3 The Connection to AI: The “Selective” Scan

This mechanism of gating information based on reliability is exactly what distinguishes **Mamba** from traditional RNNs [4] [3].

In a standard RNN, the model updates its memory with every input uniformly. Mamba, however, learns a **dynamic Kalman Gain** for each token.

- If the token is “Um...” (noise), the model sets the gain near 0, preserving its existing state.
- If the token is “NOT” (crucial signal), the model sets the gain near 1, drastically updating its state to reflect the context shift.

This **Selectivity**, knowing what to ignore is why Mamba can process massive sequences without getting overwhelmed by noise.

4 The Leap to AI: Trajectories of Thought

We have spent the last few pages landing rockets. But how does this help us build a better ChatGPT? It turns out that **Language is a Trajectory**.

- A rocket follows a path through physical space (Position, Velocity).
- A sentence follows a path through “Semantic Space” (Subject, Verb, Object).
- “The quick brown fox...” is a trajectory that must land on “jumps”.

For years, the **Transformer** (the “T” in GPT) has been the dominant way to model this path. However, it suffers from a fatal flaw: it is bad at handling “state.”

4.1 The Memory Bottleneck: $O(N^2)$ vs. $O(N)$

Imagine calculating a rocket’s position at $t = 100$. A Transformer calculates the relationship between the current moment and *every* past moment simultaneously (Attention Mechanism). This results in **Quadratic Complexity** ($O(N^2)$). If you double the length of the input, the computation becomes 4x harder. This is why ChatGPT has a “context limit”, it simply runs out of memory.

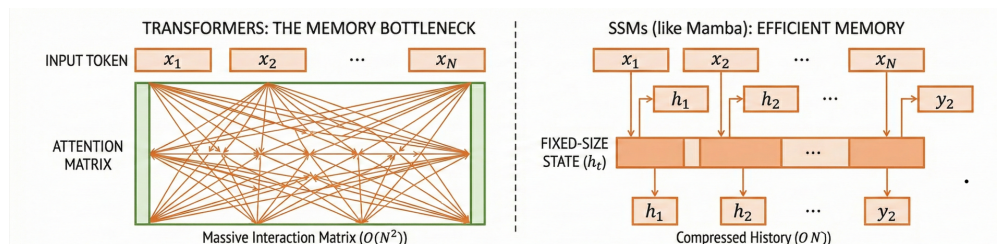


Figure 6: The Memory Bottleneck. Transformers (left) store a massive interaction matrix ($O(N^2)$). SSMs like Mamba (right) compress history into a fixed-size state ($O(N)$). [3]

Mamba flips the script. It treats language modeling exactly like our rocket landing. Instead of remembering the *entire history*, it maintains a **hidden state** (h_t), a compressed summary of the

context. When a new word arrives, Mamba updates this state using the **exact same** discrete control equations we just derived:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t \quad (5)$$

Because this is a recursive update, the complexity is **Linear** ($O(N)$). To generate the 1000th word, Mamba doesn't reread the first 999; it simply updates the state from step 999.

4.2 The Innovation: Selective State Spaces

The key innovation that separates Mamba from old-school RNNs is **Selectivity**. In a standard control system, the matrices A, B, C are constant. In Mamba, they become functions of the input:

$$B = f(x_t), \quad C = f(x_t), \quad \Delta = f(x_t)$$

This allows the model to “choose” what to remember and what to ignore, effectively learning a **dynamic Kalman Gain** for every token.

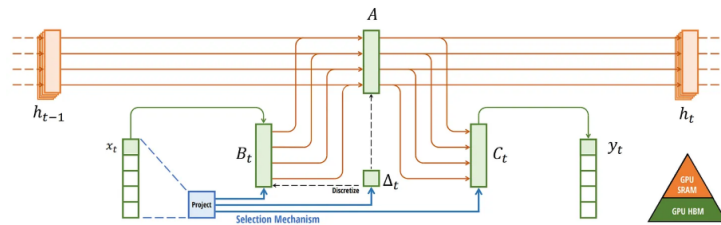


Figure 7: The Mamba Block. Parameters Δ, B, C are derived from the input, allowing the model to dynamically gate information. [3]

4.3 Applications: The Mamba Explosion

The linear efficiency of Mamba has triggered a renaissance in AI, solving problems that were computationally intractable just two years ago.

4.3.1 Long-Context Processing

Since Mamba's memory footprint is fixed, it can process sequences of effectively infinite length.

- **Genomics:** Modeling DNA sequences with millions of base pairs to predict gene function.
- **Code Generation:** Understanding entire software repositories at once rather than single files.

4.3.2 Computer Vision: Scanning the Spatial World

The challenge for SSMs is that images are 2D (spatial), while SSMs are 1D (temporal). Recent architectures like **Vision Mamba (Vim)** and **VMamba** solve this using **Cross-Scan Mechanisms**.

Instead of scanning pixels left-to-right, these models scan images in four directions (corners to center) simultaneously. This allows the 1D state equation to capture 2D global context like the shape of an object drastically faster than Vision Transformers (ViTs) [9] [10].

This efficiency allows us to deploy powerful encoders on **edge devices** without the heavy computational cost of Transformers, proving that 1960s control theory is indeed the key to the 2020s intelligence quest.

5 Conclusion: The Universal Equation

The beauty of engineering is that mathematics is universal. The same matrix operations that guided Apollo 11 to the Sea of Tranquility are now generating code, diagnosing diseases, and recognizing faces in your photo gallery.

For decades, we treated Control Theory and Artificial Intelligence as separate worlds. One was for machines that move, the other was for machines that think. Mamba has shattered that wall. It has shown us that intelligence is fundamentally about **State Estimation**, the ability to filter noise, update a mental model, and predict the future.

For us at UoP, the lesson is clear: **Don't ignore those derivations you struggle with in control modules, they are not meant to be forgotten. You aren't just learning how to control a motor, you are learning the mechanics of the next generation of intelligence.**

References

- [1] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [2] B. D. O. Anderson and J. B. Moore, *Optimal Filtering*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1979.
- [3] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.
- [4] A. Gu, K. Goel, and C. Re, "Efficiently modeling long sequences with structured state spaces," *arXiv preprint arXiv:2111.00396*, 2021.
- [5] M. Grootendorst, "A visual guide to mamba and state space models," <https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state>, 2024, accessed: 2026-02-08.

- [6] C. Burnett, “Typical state space model diagram,” Wikimedia Commons, File:Typical_State_Space_model.png, cC0 1.0 Universal Public Domain Dedication.
- [7] P. Aimonen, “Basic concept of kalman filtering,” Wikimedia Commons, File:Basic_concept_of_Kalman_filtering.svg, cC0 1.0 Universal Public Domain Dedication.
- [8] P. Magazine, “Understanding the unseen: The kalman filter,” <https://plus.maths.org/content/understanding-unseen>, 2026, university of Cambridge, Millennium Mathematics Project.
- [9] L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang, “Vision mamba: Efficient visual representation learning with bidirectional state space model,” *arXiv preprint arXiv:2401.09417*, 2024.
- [10] Y. Liu, Y. Tian, Y. Zhao, H. Yu, L. Xie, Y. Wang, Q. Ye, and Y. Liu, “Vmamba: Visual state space model,” *arXiv preprint arXiv:2401.10166*, 2024.